

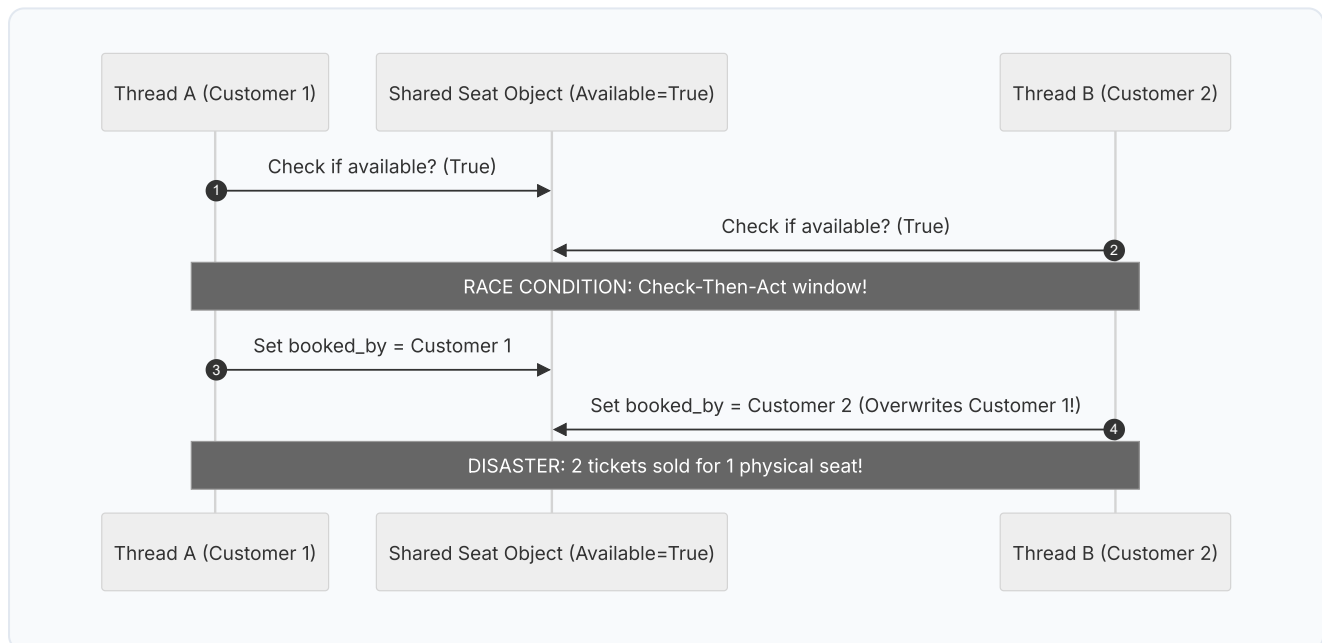
Chapter 07: Concurrency Stress-Testing & Race Condition Labs

The Flaw of "Paper" Thread-Safety In an LLD interview, drawing a class diagram and saying *"We will use locks to prevent race conditions"* sounds easy. But in production, poorly placed locks either fail to protect critical sections (leading to double-booking and financial corruption) or cause catastrophic lock contention that destroys application throughput.

This laboratory demonstrates a live multi-threaded race condition, proves how double-booking occurs at the machine level, and benchmarks three industrial concurrency strategies: **Pessimistic Locking**, **Reentrant Locks**, and **Optimistic Concurrency Control (OCC)**.

1. The Disaster: Reproducing the Double-Booking Bug

Consider a movie ticket inventory service. There is **exactly 1 VIP seat** remaining. 50 concurrent customer threads simultaneously click "Purchase".



The Unsafe Implementation

Execution Output:

14 different customers received confirmation for the same seat!

2. Solution 1: Pessimistic Locking (`threading.Lock`)

The simplest and most robust defense against race conditions is a **Mutual Exclusion Lock (Mutex)**. We wrap the entire "Check-Then-Act" compound operation into an atomic critical section.

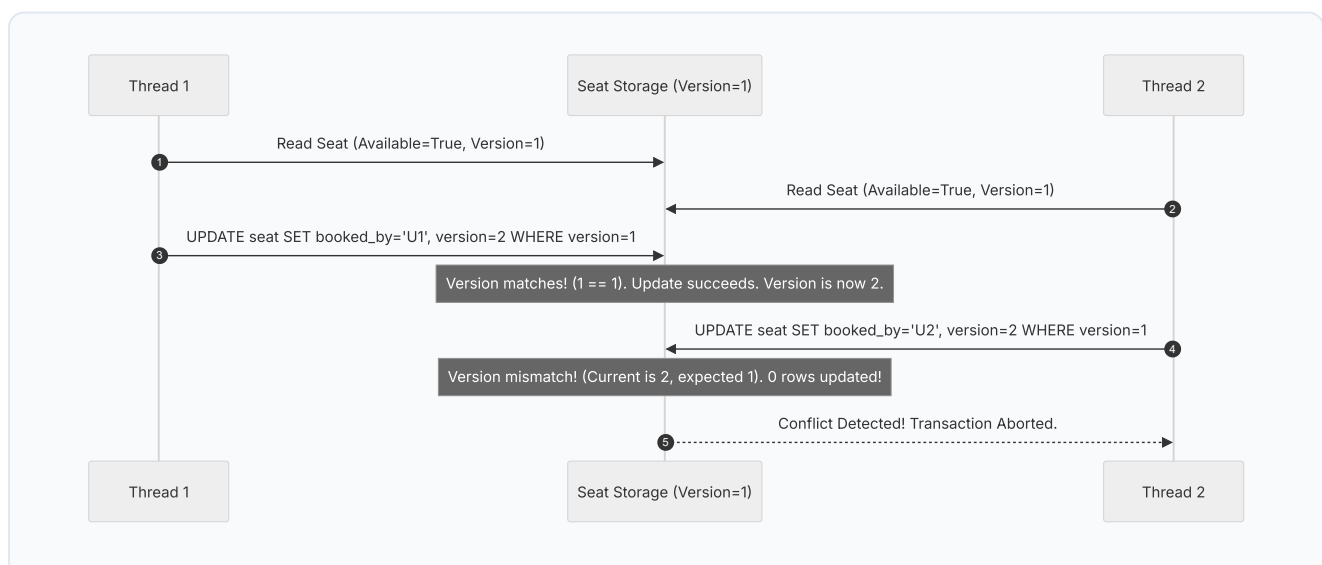
Why Reentrant Locks (`RLock`) Matter

If a method acquires `self._lock` and then internally calls another helper method on `self` that *also* acquires `self._lock`, standard `threading.Lock` causes a self-deadlock. An `RLock` allows the **same thread** to acquire the lock multiple times recursively.

3. Solution 2: Optimistic Concurrency Control (OCC)

Pessimistic locking works, but in high-traffic read-heavy systems, holding locks creates severe bottlenecks.

Optimistic Concurrency Control (OCC) assumes conflicts are rare. Instead of locking before reading, each record has a **version number**. When updating, the write succeeds **only if the version number has not changed**.



Python Implementation of In-Memory OCC

4. Benchmark & Strategy Comparison Matrix

When we run our 50-thread stress test across all three implementations:

Concurrency Pattern	Thread Safe?	Lock Overhead	Throughput Under High Contention	Best Use Case
Unsafe (No Locks)	✗ No (Double-booking)	None	N/A (Corrupts Data)	Never in concurrent environments
Pessimistic (Lock)	✓ Yes	High (Threads queue up)	Predictable, but serialized	Write-heavy operations with high contention (Concert ticket drops)
Optimistic (OCC)	✓ Yes	Very Low (Lock-free reads)	High for reads; High abort rate for writes	Read-heavy systems with low collision probability (E-commerce catalog)
Distributed Redis Lock	✓ Yes	Network round-trip	Moderate	Multi-node microservices across separate machines

Senior Interview Tip

In an LLD interview, when the interviewer asks: "How will you handle 10,000 users booking tickets at the same second?", give this two-tiered answer:

1. "At the database level, I will use **Optimistic Concurrency Control** via an atomic `UPDATE seats SET status='BOOKED', version=v+1 WHERE id=:id AND version=:v` to prevent holding long-lived DB transactions."
2. "At the application gateway layer, I will use a **Redis Distributed Lock with a short TTL (Redlock)** to prevent 10,000 requests from hitting the database simultaneously in the first place."

Further Reading

- [threading module](#)
- [concurrent.futures](#)
- [pytest documentation](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

▼ 1. How do you reproduce a race condition reliably?

Increase contention (many threads, many iterations), add small sleeps/yields at the racy point, and assert an invariant at the end.

▼ 2. A test passes 100 times then fails once. What does that suggest?

A non-deterministic race or ordering bug; passing runs do not prove safety, so examine shared state and synchronisation.

▼ 3. What does `threading.Lock` not protect against?

Logic races across separate critical sections (check-then-act split into two locked regions) and deadlocks.

▼ 4. Name a tool or technique to find data races.

Stress tests with invariants, thread sanitizers in native code, and code review of shared mutable state.